Session 13: File Handling — Notes & Code (C Language)

Mastering File Streams, Modes (r, w, a), I/O Functions, and Search Algorithms

October 11, 2026

Contents

1 Topic Notes **2**
- 1.1 What is File Handling? . . . 2
- 1.2 Common File Functions . . . 2
- 1.3 File Modes . . . 2
- 1.4 Key Technical Points . . . 2

2 Task 1: Write Top 3 Songs into playlist.txt (Mode: `"w"`) **2**
- 2.1 Code . . . 2
- 2.2 File Content (`playlist.txt`) . . . 3

3 Task 2: Read playlist.txt Line by Line (Mode: `"r"`) **4**
- 3.1 Code . . . 4
- 3.2 Output . . . 4

4 Task 3: Add 2 More Songs (Mode: `"a"` — Append) **5**
- 4.1 Code . . . 5
- 4.2 Updated File Content (`playlist.txt`) . . . 5

5 Task 4: Print Only Songs Containing `"love"` (Case-Insensitive) **6**
- 5.1 Code . . . 6
- 5.2 Sample `playlist.txt` for Search . . . 6
- 5.3 Output . . . 7

6 Quick Revision Summary **8**

1 Topic Notes

1.1 What is File Handling?

- File handling lets a program **store data permanently** on disk and read it back later.
- In C, we use the `FILE` pointer and functions from `<stdio.h>`.

1.2 Common File Functions

Function	Purpose
`fopen("file", "mode")`	Opens a file; returns a `FILE*` pointer
`fprintf(fp, ...)`	Writes formatted data to a file
`fgets(buffer, size, fp)`	Reads one line from a file
`fclose(fp)`	Closes the file and saves changes

Table 1: Common File Handling Functions in C

1.3 File Modes

Mode	Meaning	Behavior
`"r"`	Read	File must exist; opens for reading only
`"w"`	Write	Creates file; **erases existing content**
`"a"`	Append	Adds data at the **end**; keeps existing content

Table 2: File Opening Modes

1.4 Key Technical Points

- Always check if `fopen()` returns `NULL` (file may not exist / can't open).
- Always `fclose()` the file after use to ensure proper saving and resource release.
- `"w"` deletes old data; use `"a"` to preserve it.
- `fgets()` reads line by line, including the newline character `\n`.

2 Task 1: Write Top 3 Songs into playlist.txt (Mode: `"w"`)

2.1 Code

```c
#include <stdio.h>

int main() {
    FILE *fp;
```

```c
    // Open file in write mode (creates file, erases old content)
    fp = fopen("playlist.txt", "w");

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    fprintf(fp, "Tum Hi Ho\n");
    fprintf(fp, "Perfect\n");
    fprintf(fp, "Blinding Lights\n");

    fclose(fp);

    printf("Top 3 songs written to playlist.txt successfully!\n");

    return 0;
}
```

Listing 1: Writing Data to a File in Write Mode

2.2 File Content (`playlist.txt`)

```
Tum Hi Ho
Perfect
Blinding Lights
```

3 Task 2: Read playlist.txt Line by Line (Mode: "r")

3.1 Code

```c
#include <stdio.h>

int main() {
    FILE *fp;
    char song[100];

    // Open file in read mode
    fp = fopen("playlist.txt", "r");

    if (fp == NULL) {
        printf("Error: playlist.txt not found!\n");
        return 1;
    }

    printf("----- My Playlist -----\n");
    while (fgets(song, sizeof(song), fp) != NULL) {
        printf("%s", song);   // song already contains '\n'
    }

    fclose(fp);

    return 0;
}
```

Listing 2: Reading Line-by-Line Using fgets

3.2 Output

```
----- My Playlist -----
Tum Hi Ho
Perfect
Blinding Lights
```

Note: `fgets()` returns `NULL` at the end of the file (EOF), which safely terminates the loop.

4 Task 3: Add 2 More Songs (Mode: "a" — Append)

4.1 Code

```c
#include <stdio.h>

int main() {
    FILE *fp;

    // Open in append mode -- existing content is preserved
    fp = fopen("playlist.txt", "a");

    if (fp == NULL) {
        printf("Error opening file!\n");
        return 1;
    }

    fprintf(fp, "Levitating\n");
    fprintf(fp, "Believer\n");

    fclose(fp);

    printf("Two new songs appended to playlist.txt!\n");

    return 0;
}
```

Listing 3: Appending Data Using Mode a

4.2 Updated File Content (playlist.txt)

```
Tum Hi Ho
Perfect
Blinding Lights
Levitating
Believer
```

Note: If "w" mode had been used here instead of "a", the first three songs would have been completely **erased**. Append mode preserves prior records.

5 Task 4: Print Only Songs Containing "love" (Case-Insensitive)

5.1 Code

```c
#include <stdio.h>
#include <string.h>
#include <ctype.h>

// Convert a string to lowercase (for case-insensitive comparison)
void toLowerCase(char *str) {
    for (int i = 0; str[i] != '\0'; i++) {
        str[i] = tolower(str[i]);
    }
}

int main() {
    FILE *fp;
    char song[100];
    char lowerSong[100];

    fp = fopen("playlist.txt", "r");

    if (fp == NULL) {
        printf("Error: playlist.txt not found!\n");
        return 1;
    }

    printf("Songs containing 'love':\n");
    while (fgets(song, sizeof(song), fp) != NULL) {
        strcpy(lowerSong, song);   // copy before modifying
        toLowerCase(lowerSong);    // make lowercase

        // Check if 'love' is inside the song name
        if (strstr(lowerSong, "love") != NULL) {
            printf("%s", song);    // print original song name
        }
    }

    fclose(fp);

    return 0;
}
```

Listing 4: Case-Insensitive Substring Search in Files Using strstr

5.2 Sample `playlist.txt` for Search

```
Tum Hi Ho
Perfect
Blinding Lights
Levitating
Believer
All of Me
Love Story
```

5.3 Output

```
Songs containing 'love':
Love Story
```

Technical Takeaways:

- `strstr(text, word)` searches for a substring inside a larger string.
- Since `strstr()` is case-sensitive, lowering both strings ensures matching works for `Love`, `LOVE`, and `love`.

6 Quick Revision Summary

Concept / Operation	Description / Syntax
File Opening	`FILE *fp = fopen("file.txt", "mode");`
Null Check	Always verify `if (fp == NULL)` before file operations
Write Mode (`"w"`)	Creates file or overwrites existing content completely
Read Mode (`"r"`)	Opens file for reading; fails if file does not exist
Append Mode (`"a"`)	Adds data to the end without deleting prior content
Line Reading	`fgets(buffer, size, fp)`
File Closing	`fclose(fp);` (mandatory to save and flush buffers)
Substring Matching	`strstr()` combined with lowercase conversion for search

Table 3: File Handling Quick Reference

Core Takeaway: File streams allow C applications to persist data beyond runtime. Choosing the right mode (`r`, `w`, `a`) ensures data safety and expected read/write behavior.